Python Developer Internship Assignment

Submitted by: Tanmay Itape

Date: June 12, 2026

Submitted to: khushi@tradexa.co.in

1. Project Overview

A Django 6.x web application built with two apps — users and products — each connected to its own SQLite database via a custom database router. The project demonstrates custom user authentication, post creation with @login_required, a ForeignKey without database constraint, admin customization, and public product listing.

2. Tech Stack

- Backend: Python 3, Django 6.x
- Frontend: Bootstrap 5 (CDN)
- Database: SQLite (two separate files)
- Configuration: python-decouple (environment variables)
- Version Control: Git & GitHub

3. Folder Structure

```
tradexa_assignment/
├── config/
│   ├── settings.py
│   ├── urls.py
│   ├── routers.py
│   └── ...
├── users/
│   ├── models.py
│   ├── views.py
│   ├── forms.py
│   ├── admin.py
│   └── templates/users/...
├── products/
│   ├── models.py
│   ├── views.py
│   ├── admin.py
│   └── templates/products/...
├── users_db.sqlite3
├── products_db.sqlite3
├── manage.py
├── requirements.txt
├── .env.example
└── README.md
```

4. Models

User (users.models.User)

- Inherits from AbstractUser
- Provides username, first_name, last_name, email, password

Post (users.models.Post)

- user: ForeignKey to User with db_constraint=False, on_delete=CASCADE
- text: TextField
- created_at, updated_at: auto-managed DateTime fields

Product (products.models.Product)

- name: CharField
- weight: DecimalField (kg)
- price: DecimalField
- created_at, updated_at

5. Database Router Explanation

- Router file: config/routers.py
- Routes users app to default database (users_db.sqlite3)
- Routes products app to products_db database (products_db.sqlite3)
- Prevents cross-database relations via allow_relation()
- Ensures migrations run only on the correct database via allow_migrate()
- Every read/write operation is directed based on app_label

6. Authentication Flow

- Custom user model set via AUTH_USER_MODEL = 'users.User'
- Django's built-in LoginView and LogoutView used
- Post creation view decorated with @login_required — unauthenticated users are redirected to login with ?next=/create/
- Session-based authentication handled by Django middleware
- Navbar shows dynamic links based on user.is_authenticated

7. Admin Customization

- All models registered with custom ModelAdmin classes
- UserAdmin inherited for user management
- PostAdmin and ProductAdmin include list_display, list_filter, search_fields
- Admin accessible only to staff users

8. Screenshots

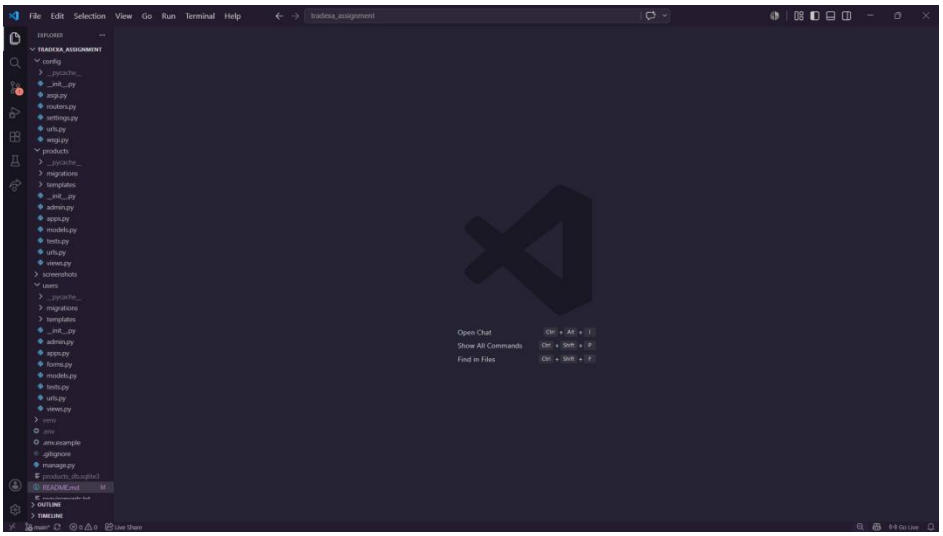

Figure 1: Project Folder Structure

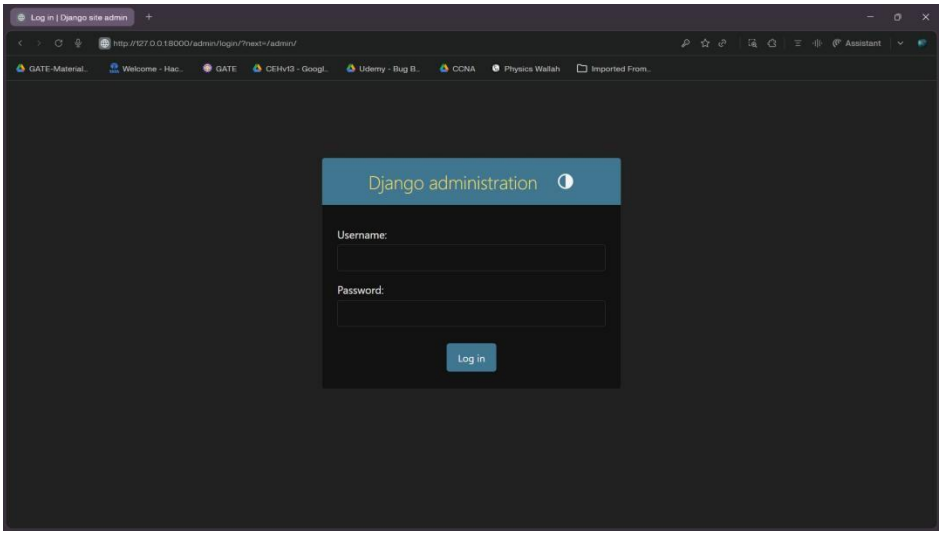

Figure 2: Admin Login Page

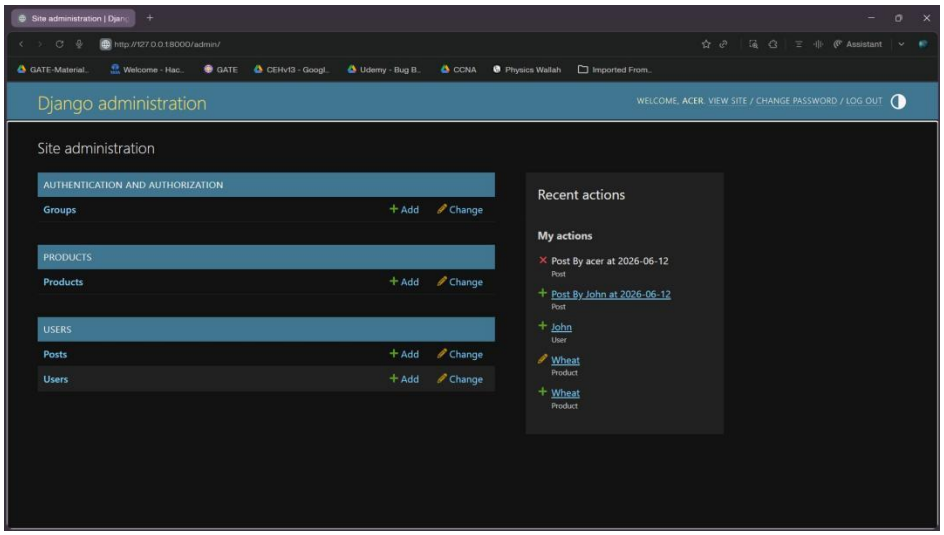

Figure 3: Admin Dashboard

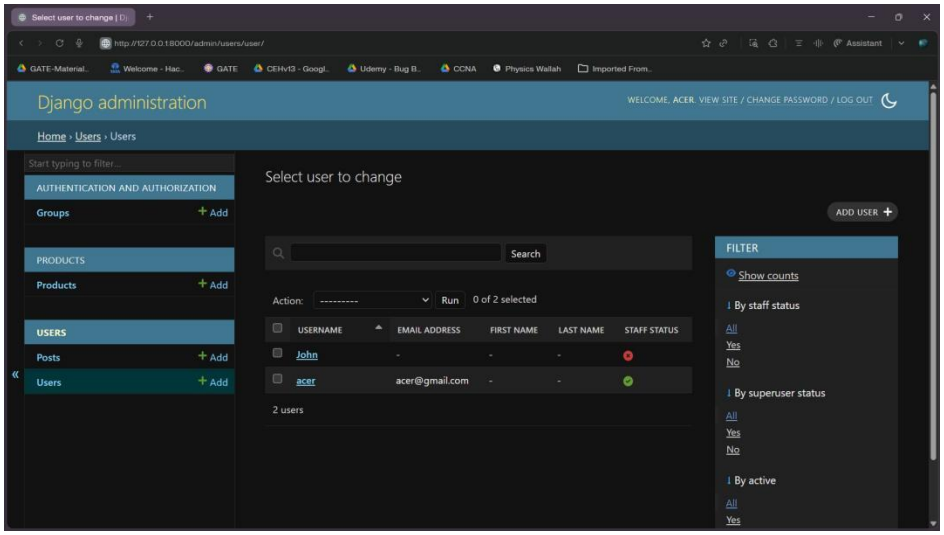

Figure 4: Users List (Admin)

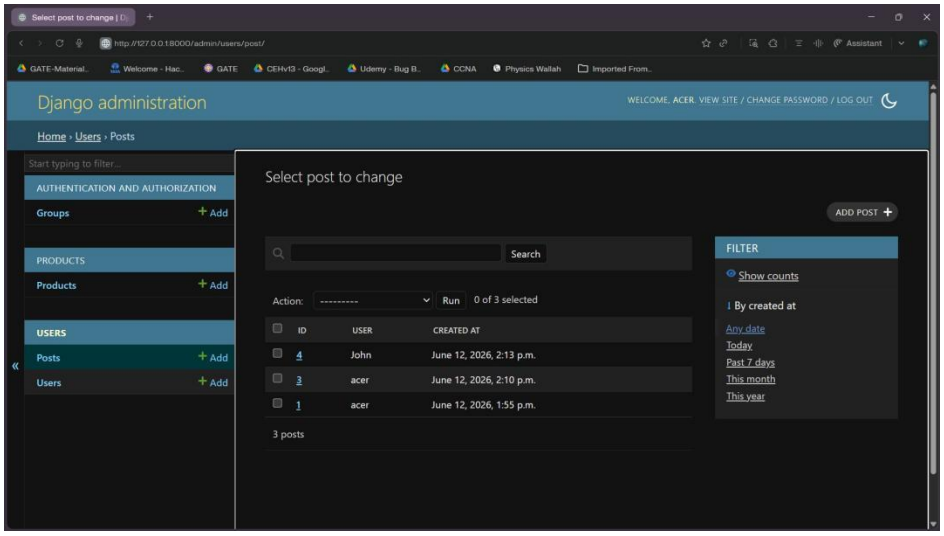

Figure 5: Posts List (Admin)

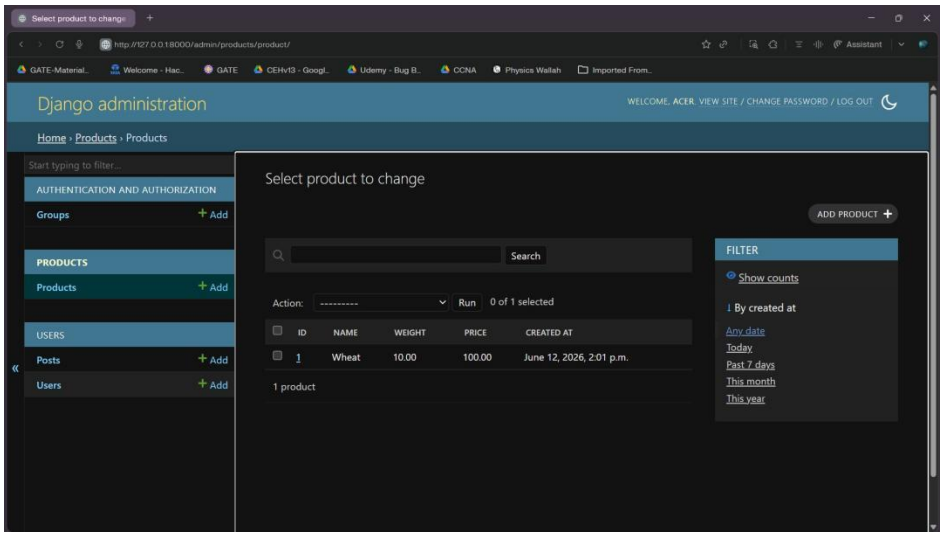

Figure 6: Products List (Admin)

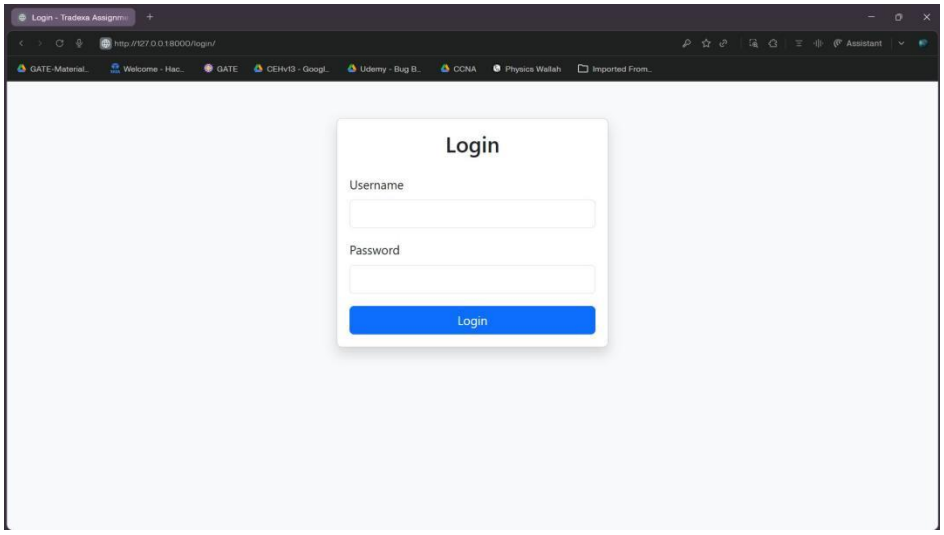

Figure 7: User Login Page

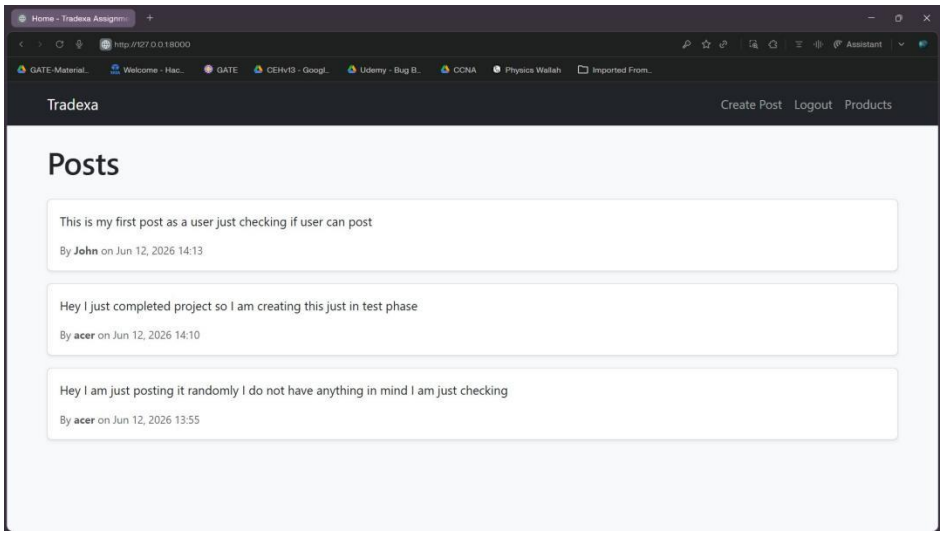

Figure 8: Home Page (Posts Feed)

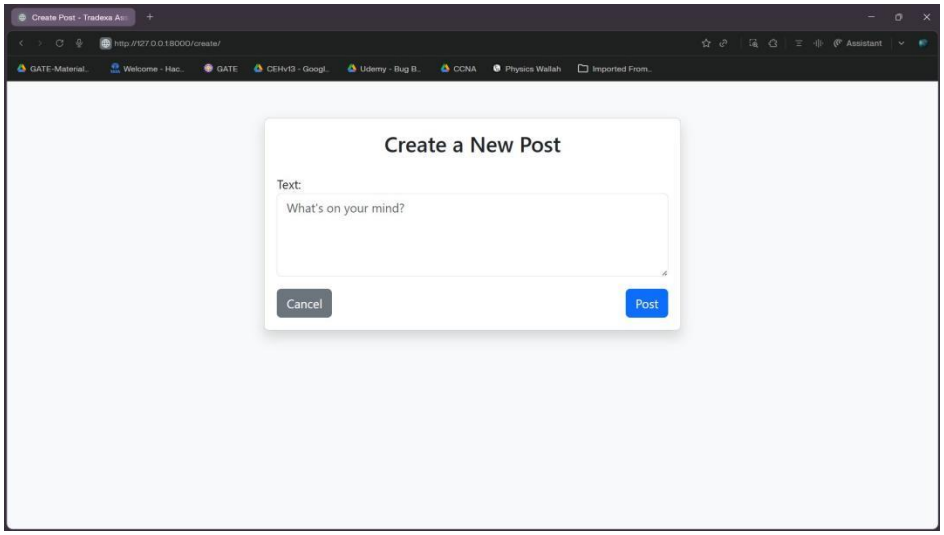

Figure 9: Create Post Page

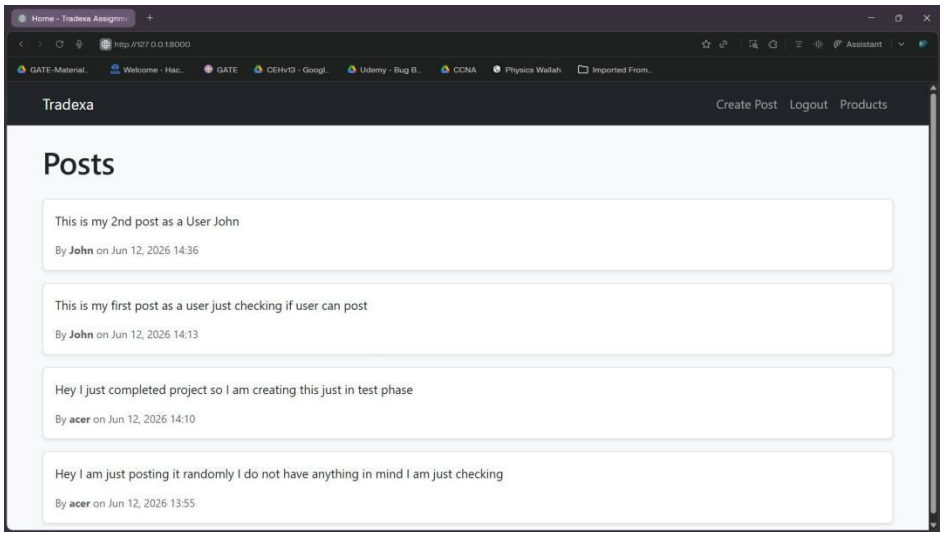

Figure 10: Successful Post Created

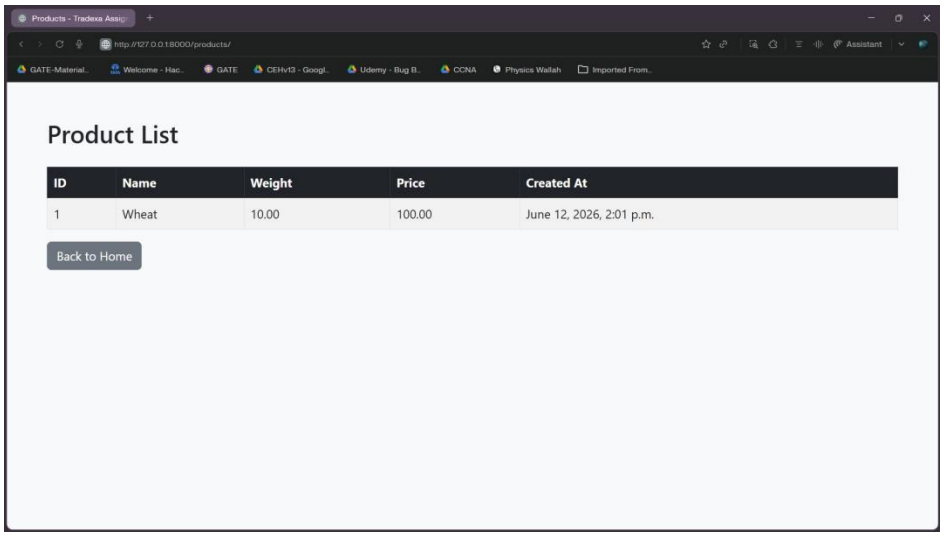

Figure 11: Public Products Page

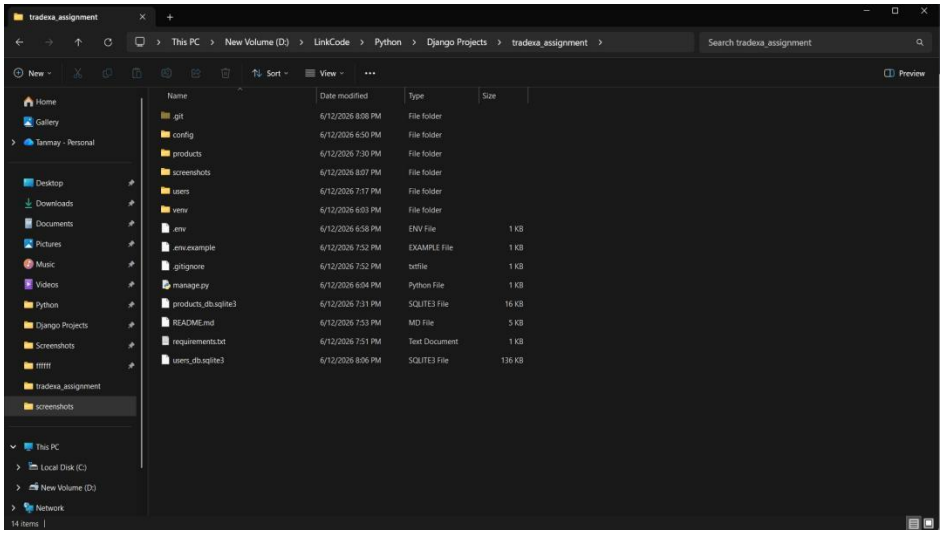

Figure 12: Database Files

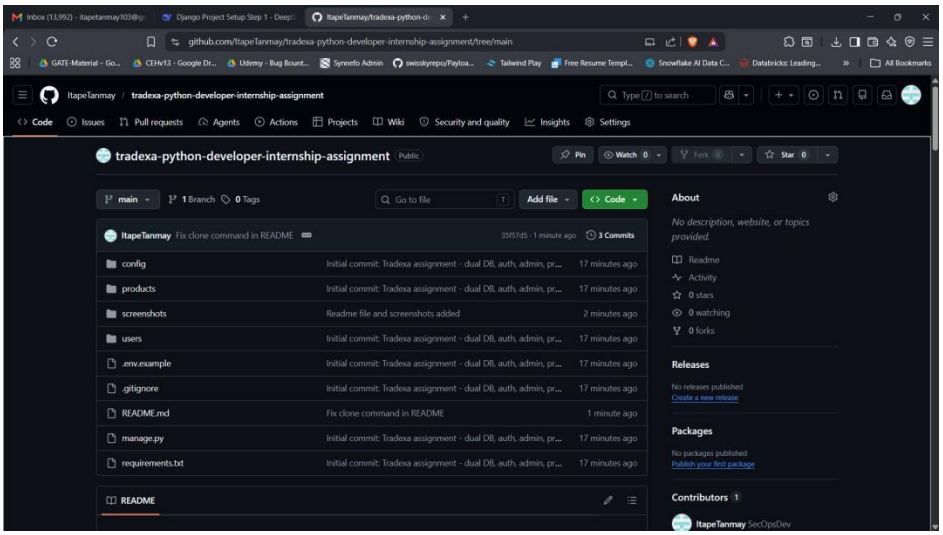

Figure 13: GitHub Repository

9. GitHub Link

https://github.com/ItapeTanmay/tradexa-python-developer-internship-assignment

10. Conclusion

This project demonstrates the ability to build a multi-app Django application with advanced configurations like dual databases, database routing, custom authentication, and admin customization. All assignment requirements have been fulfilled, and the code follows industry best practices (PEP 8, separation of concerns, secure config via environment variables).